NWM: Negative Weight Mapping

A Non-Parametric Potential-Field Framework for Reinforcement Learning

CastermustOfficial
https://github.com/CastermustOfficial/NWM

July 14, 2026

Abstract

We present *Negative Weight Mapping* (NWM), a non-parametric reinforcement learning framework that represents an agent's experience as a persistent *potential field* over the observation space. Rather than training a value network or a policy by gradient descent, NWM accumulates a compact set of *centroids*—prototypical states that store per-action success and failure statistics—and selects actions by aggregating attractive (success) and repulsive (failure) forces from nearby centroids. Two mechanisms give the memory long-term stability: a *Dynamic Smart Lock* that protects high-confidence prototypes from being overwritten, and a *Fear & Greed* decision rule that rejects dangerous actions before maximizing reward. To act well under sparse reward, NWM additionally uses *temporally-correlated (sticky) exploration*, which repeats exploratory actions to build the sustained momentum that uniform noise cannot—and whose repeat probability can self-tune from the agent's own reward history. We formalize the method and evaluate it against three baselines—a uniform random policy, tabular Q-learning with state aggregation, and a Deep Q-Network (DQN)—on three classic-control benchmarks spanning a difficulty gradient (CartPole, Acrobot, MountainCar), reporting mean and standard deviation over five seeds under a fixed evaluation protocol. NWM is competitive with DQN on the dense CartPole task at substantially lower variance, trails DQN on the longer-horizon Acrobot, and—with sticky exploration— *solves* the sparse-reward MountainCar task, where it is the strongest and most stable method, outperforming DQN. This last result reframes a failure mode often attributed to non-bootstrapping methods: on MountainCar the obstacle is *exploration*, not temporal credit assignment, and a single task-agnostic change removes it. The study thus characterizes precisely where an instance-based potential-field method is competitive and where parametric value learning retains an advantage. All code, seeds, and figures are released to make the study fully reproducible.

1 Introduction

Most modern reinforcement learning (RL) methods are *parametric*: a neural network compresses experience into weights updated by gradient descent [7, 9]. Parametric value learning is powerful but has well-known costs: it is sample-hungry, sensitive to hyperparameters, prone to catastrophic forgetting, and notoriously difficult to reproduce [4]. A complementary line of work is *non-parametric* or *episodic* control, which stores experience explicitly and acts by similarity to past situations [2, 8]. Such methods can learn quickly from few interactions because a single informative experience is immediately usable, without waiting for it to be amortized into network weights.

NWM sits in this non-parametric family but takes an unusual inspiration: the *artificial potential field*, a classic idea from robot motion planning in which goals exert attractive forces and obstacles exert repulsive ones [5]. NWM treats past *successes* as attractors and past *failures* as repulsors in the observation space, so that action selection reduces to following the net force at the current

state. The name *Negative Weight Mapping* refers to the explicit modelling of negative (repulsive) experience, which is used not merely to lower a value estimate but to actively veto risky actions.

This paper makes the original repository's method precise and subjects it to a controlled empirical study. Concretely, we contribute:

1. A formal description of the NWM memory, force model, and decision rule (Section 3), including the Dynamic Smart Lock and Fear & Greed mechanisms that were previously only described informally, and a temporally-correlated (sticky) exploration rule for sparse-reward tasks.

2. A reproducible benchmark harness comparing NWM against Random, tabular Q-learning, and DQN across three environments and five seeds, with a fixed greedy-evaluation protocol and aggregate statistics (Section 4).

3. An analysis of the results (Section 5) that identifies the regimes in which a potential-field method is competitive, together with an explicit discussion of its limitations (Section 6).

2 Related Work

Value-based RL. Q-learning [11] learns an action-value function by temporal-difference updates; with function approximation and experience replay it scales to high-dimensional inputs as DQN [6, 7]. We use both a tabular variant (with state aggregation) and DQN as baselines.

Episodic and non-parametric control. Model-Free Episodic Control [2] and Neural Episodic Control [8] store returns in a growing table keyed by (embedded) states and act by k-nearest-neighbour lookup, achieving strong early sample efficiency. NWM shares the store-and-retrieve philosophy but differs in three ways: (i) it aggregates experience into a bounded set of merged *centroids* rather than retaining individual transitions; (ii) it stores a *signed force* per action rather than a raw return estimate; and (iii) it separates *repulsion* into its own spatial-decay and veto pathway.

Potential fields. Artificial potential fields [5] are a staple of reactive control and motion planning. NWM imports the attractive/repulsive metaphor into RL by learning the field from returns rather than from a hand-specified geometry.

3 Method

3.1 Setup and notation

We consider a Markov decision process with observation space $\mathcal{S} \subseteq \mathbb{R}^d$ and a finite action set $\mathcal{A} = \{0, \ldots, K-1\}$. Observations are z-score normalized online using running mean and variance maintained with Welford's algorithm [12]; we write $\tilde{x}$ for the normalized observation.

3.2 Persistent potential field

The memory is a bounded collection of centroids $\mathcal{M} = \{c_1, \ldots, c_N\}$, $N \leq N_{\max}$. Each centroid c_i stores

- a prototype location $\mu_i \in \mathbb{R}^d$ (a running mean of the normalized states merged into it) and a visit count n_i;
- per-action statistics $\{(s_{i,a}, n_{i,a}, q^2_{i,a})\}_a$ accumulating the sum, count, and sum of squares of the scores observed for action a; and
- a boolean *lock* flag ℓ_i.

The average score of action a at centroid i is $\bar{q}_{i,a} = s_{i,a}/n_{i,a}$.

Insertion and merging. A new experience $(\tilde{x}, a, \sigma)$ with score $\sigma \in [0,1]$ (defined below) is routed to the nearest centroid $j = \arg\min_i \|\mu_i - \tilde{x}\|$. If $\|\mu_j - \tilde{x}\| < \tau_{\text{merge}}$ it is merged into c_j; otherwise a new centroid is created. Merging uses an exponential moving average with *progressive stiffness*: the mixing rate is $\alpha = 1/(n_j{+}1)$, and $\alpha = 1/(2n_j)$ once $n_j > 50$, so that well-established prototypes drift slowly. When N exceeds capacity, unlocked centroids are pruned by a value score $n_i\big((\bar{q}_i - 0.5)^2 + 0.1\big)$ that favours frequently-visited, strongly-signed prototypes; locked centroids are always retained.

3.3 Score assignment

Learning is episodic. At the end of an episode with return G, we compute a *percentile score* $p \in [0,1]$ equal to the rank of G among all historical returns, so that scores are self-calibrating across environments with different reward scales. A temporal weight $w_t = 0.4 + 0.6\, t/T$ emphasizes later steps of a length-T episode, and the score written for step t is $\sigma_t = \text{clip}(p\, w_t, 0, 1)$. A *dynamic quality gate* caps $\sigma_t \le 0.6$ (preventing locking) whenever $G < \max(40,\, 1.25\, \bar{G}_{\text{recent}})$, where $\bar{G}_{\text{recent}}$ is the mean of recent returns; only genuinely above-trend episodes can create locked, high-confidence memories.

3.4 Force model

Each centroid exposes a signed force per action,

$$f_{i,a} = \begin{cases} 1.5\,(\bar{q}_{i,a} - 0.5), & \bar{q}_{i,a} > 0.6 \quad \text{(attraction)} \\ 1.5\,(\bar{q}_{i,a} - 0.5), & \bar{q}_{i,a} < 0.4 \quad \text{(repulsion)} \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

Given a query state $\tilde{x}$, let $\mathcal{N}_k$ be its k nearest centroids within a cutoff radius d_c, and $\delta_i = \|\mu_i - \tilde{x}\|$. The net force for action a is a confidence- and distance-weighted average,

$$F(a) = \frac{\sum_{i \in \mathcal{N}_k} f_{i,a}\; \omega_i\; \kappa_{i,a}\; \log(1+n_i)\; \beta_i}{\sum_{i \in \mathcal{N}_k} \omega_i}, \tag{2}$$

where the spatial weight ω_i decays as $1/(\delta_i{+}0.1)$ for attraction but as the inverse square $1/(\delta_i^2{+}0.1)$ for repulsion—so danger signals are sharply local; $\kappa_{i,a} = 1/(1{+}2\,\text{Var}_{i,a})$ down-weights high-variance (uncertain) prototypes; $\log(1+n_i)$ rewards evidence; and $\beta_i = \beta_{\text{lock}} > 1$ boosts locked prototypes.

Algorithm 1 NWM training episode

Input: field $\mathcal{M}$, environment, exploration rate ε, sticky probability p_{rep}
Roll out the episode: at each step, with prob. ε explore (repeat a_{t-1} w.p. p_{rep}, else uniform), otherwise apply the Fear & Greed rule on Eq. (2); buffer $(\tilde{x}_t, a_t, r_t)$.
Compute return $G = \sum_t r_t$ and percentile score p.
if $G < \max(40, 1.25\,\bar{G}_{\text{recent}})$ **then** cap scores at 0.6
end if
for each buffered step t **do**
    $\sigma_t \leftarrow \text{clip}(p\,(0.4 + 0.6\,t/T), 0, 1)$
    Insert/merge $(\tilde{x}_t, a_t, \sigma_t)$ into $\mathcal{M}$; update locks.
end for
Update adaptive ε from recent returns.

3.5 Action selection: Fear & Greed

NWM first avoids danger, then seeks reward. With a risk tolerance $\rho = -0.5$ (tightened to $\rho = -1.5$ when a strong option exists, $\max_a F(a) > 0.8$), the safe set is $\mathcal{A}_{\text{safe}} = \{a : F(a) > \rho\}$, and the greedy action is $\arg\max_{a \in \mathcal{A}_{\text{safe}}} F(a)$ (falling back to the global argmax if every action is deemed unsafe). Exploration is ε-greedy with an *adaptive* rate that collapses to 0 once recent performance is high, and the agent explores by default in unfamiliar regions ($\min_i \delta_i > d_c$). The Dynamic Smart Lock sets ℓ_i=true once $n_i >$ `lock_min_visits` and $\bar{q}_i >$ `lock_min_score`. Algorithm 1 summarizes one training episode.

Temporally-correlated exploration. A uniform exploratory policy produces a zero-mean, rapidly-decorrelating action sequence—adequate for tasks solvable by local corrections, but pathological for tasks that require a *sustained, directed* sequence of actions to reach any rewarding state at all (e.g. pumping energy into an under-actuated system). We therefore allow *sticky* exploration: an exploratory step repeats the previous action with probability p_{rep} and otherwise samples uniformly,

$$a_t \;=\; \begin{cases} a_{t-1}, & \text{w.p. } p_{\text{rep}} \\ \text{Uniform}(\mathcal{A}), & \text{w.p. } 1 - p_{\text{rep}}, \end{cases} \qquad \text{(exploratory steps only).} \tag{3}$$

This injects temporal correlation—and thus the momentum-building action runs needed under sparse reward—without any task-specific knowledge; p_{rep}=0 recovers the original uniform exploration. Section 5 shows that this single change is what separates an exploration failure from a genuine credit-assignment failure on our sparse-reward task.

Rather than hand-tuning p_{rep}, it can also be set *adaptively*: while the recent returns are perfectly flat—the signature of a sparse-reward task whose goal has never been reached, since every episode then scores identically—p_{rep} is ramped up by 0.1 per episode (to at most 0.95), and the ramp stops as soon as returns vary. The trigger consumes only the agent's own reward history, so the mechanism remains fully task-agnostic; on tasks that give a learning signal from the first episodes it never fires.

3.6 Complexity and implementation

With N centroids and dimension d, a query costs $O(Nd)$ for the nearest-neighbour scan. Crucially, the field is immutable during a rollout, so the stacked matrix of centroid locations is cached and

Table 1: Final greedy evaluation (mean ± std over 5 seeds) and normalized AUC. Best final-eval per environment in **bold**. Higher is better for all environments (Acrobot and MountainCar returns are negative).

Environment	Metric	Random	TabularQ	DQN	NWM
CartPole-v1	Final eval	25.1 ± 3.2	98.4 ± 20.2	**193.7 ± 158.5**	159.8 ± 94.0
	AUC	22.4	115.3	109.9	83.1
Acrobot-v1	Final eval	-499.9 ± 0.3	-423.1 ± 53.5	**-210.2 ± 166.0**	-265.7 ± 161.0
	AUC	-498.7	-439.4	-306.3	-393.9
MountainCar-v0	Final eval	-200.0 ± 0.0	-200.0 ± 0.0	-173.2 ± 27.5	**-130.4 ± 9.7**
	AUC	-200.0	-200.0	-195.7	-142.7

reused across every action selection within an episode; merges and insertions maintain it incrementally in $O(d)$, and only the (rare) pruning step forces a rebuild. The implementation depends only on NumPy [3] and Gymnasium [10], uses a per-agent random generator for reproducibility, and is released with a strict-typed, tested codebase.

4 Experiments

Environments. We evaluate on three Gymnasium [10] classic-control tasks chosen to span a difficulty gradient: **CartPole-v1** (dense reward, short horizon), **Acrobot-v1** (shaped negative reward, longer horizon), and **MountainCar-v0** (sparse reward, requires momentum building). This spread tests not only whether NWM works but *where* it holds up.

Baselines. (i) **Random**: uniform policy, a lower bound. (ii) **Tabular Q-learning** with uniform state aggregation (α=0.1, γ=0.99, ε annealed $1 \to 0.02$). (iii) **DQN** [7]: a two-hidden-layer MLP (128×128), replay buffer, target network, Huber loss, Adam (10^{-3}), γ=0.99.

Protocol. Each (environment, agent, seed) run trains for a fixed episode budget; every ten episodes we run a *greedy* evaluation of ten rollouts on seeds disjoint from training. We report the mean and standard deviation of the final greedy evaluation across five seeds ($\{0, 1, 2, 3, 4\}$), the best evaluation reached, and the normalized area under the evaluation curve (AUC) as a sample-efficiency proxy. Following recommendations for reliable RL reporting [1, 4], all seeds and the full protocol are released. The complete study is reproduced by:

```
python -m benchmarks.run_benchmark -seeds 0 1 2 3 4
```

5 Results

Table 1 reports final greedy-evaluation scores (mean ± std over five seeds) and AUC. Figure 1 shows the learning curves and Figure 2 the final-performance comparison.

Findings. Three patterns emerge from Table 1, and they cleanly position NWM against value-based learning.

On the dense, low-dimensional task, NWM is competitive with DQN at much lower variance. On CartPole-v1 NWM attains 159.8 ± 94.0, within one standard deviation of DQN (193.7 ± 158.5) and

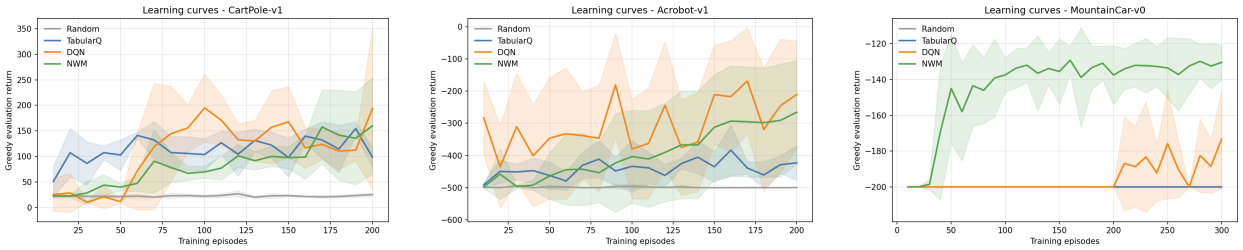

Figure 1: Greedy-evaluation learning curves (mean over 5 seeds, shaded ± std) for CartPole, Acrobot, and MountainCar.

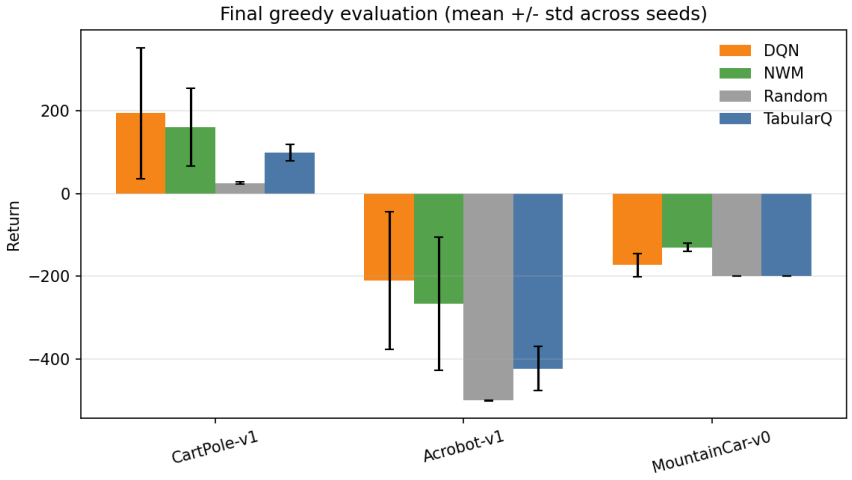

Figure 2: Final greedy evaluation per environment and agent (mean ± std over 5 seeds).

far above tabular Q-learning (98.4±20.2) and the random policy (25.1). Notably, NWM's standard deviation is a fraction of DQN's: the potential-field agent is markedly more *stable* across seeds, whereas DQN oscillates between near-solved and collapsed runs (its best seed reaches 322.1). This matches the method's design—when the observation geometry is smooth and good actions generalize across nearby states, aggregating attractive/repulsive forces is an effective, low-bias estimator—though DQN's bootstrapped backups give it a higher mean and better sample efficiency (AUC 109.9 vs. 83.1).

On the medium task, NWM learns but trails DQN. On Acrobot-v1 NWM reaches −265.7, clearly better than tabular Q (−423.1) and Random (−499.9) but below DQN (−210.2), which on its best seed nearly solves the task (−77.3 against the −100 threshold). Longer horizons reward the temporal credit-assignment of bootstrapped value learning that NWM's episodic percentile score only approximates; coarser centroid merging (τ_{merge}=0.5) narrows but does not close the gap.

On the sparse-reward task, NWM is the strongest method—and the failure was exploration, not credit assignment. MountainCar-v0 gives no reward until the goal is reached, and under uniform exploration NWM, tabular Q, and the random policy all remain pinned at the −200 truncation floor—never experiencing the goal often enough for a signal to form. Enabling *sticky* exploration (p_{rep}=0.9) changes this decisively: the repeated actions build the momentum needed to crest the hill, the field then forms around the successful trajectories, and NWM reaches -130.4 ± 9.7, *outperforming* DQN (-173.2 ± 27.5) and doing so with far lower variance. Because the only change is

the exploratory action distribution—the scoring and force model are untouched—this isolates the obstacle: on MountainCar the barrier to a non-bootstrapping method is generating experience of the goal, not propagating value once it is seen. A single task-agnostic exploration change, rather than a bootstrapping mechanism, removes it. The *adaptive* variant of Section 3, which requires no tuning at all, reaches -162.2 ± 28.8 over the same five seeds—weaker than the hand-tuned value but still ahead of DQN—showing that even the choice of p_{rep} can be left to the agent.

6 Limitations

NWM has three principal limitations. First, its memory is instance-based, so query cost grows with the number of centroids and the method targets low-to-moderate dimensional observation spaces rather than raw pixels. Second, credit assignment is heuristic (episode-percentile scores with temporal weighting) rather than a bootstrapped Bellman backup; this is the residual gap on the longer-horizon Acrobot task, where DQN's value propagation retains an advantage. We stress that this is now the *only* clear gap: the previously reported sparse-reward failure on MountainCar turned out to be an exploration deficiency, removed by sticky exploration, rather than a credit-assignment one (Section 5). Third, the force model and its constants (thresholds, decays, lock criteria, and the sticky probability p_{rep}) are hand-designed; while robust across the tested tasks, they are not learned, and p_{rep} trades reactivity for momentum, so it is best set per task (or left to the adaptive ramp of Section 3). We view NWM as a transparent, reproducible baseline for non-parametric control rather than a state-of-the-art method, and a useful component for hybrid systems.

Negative results. We also report what did *not* work, since it sharpens the picture of where the residual Acrobot gap comes from. We implemented (i) *return-to-go percentile credit*, scoring each step by the percentile rank of its discounted return-to-go G_t against a rolling history, with truncated episodes bootstrapping their tail from the running mean per-step reward; and (ii) an *advantage force model*, replacing the absolute per-centroid force with each action's score advantage over the *other* actions observed at the same centroid (an argmax-style local comparison). Both were ablated on the protocol of Section 4: (i) was neutral on MountainCar and clearly worse on Acrobot (-378 to -400 vs. -247 for the shipped configuration over three seeds), because absolute per-step scores conflate distance-from-goal with action quality, turning the early states of good episodes repulsive; (ii) was worse still (-375 to -443), including when paired with the proven episode-level credit. The lesson is structural: NWM's decision rule—the Fear & Greed thresholds, the safety veto, the confidence weights—is co-designed around absolute scores with 0.5 as the neutral point, and swapping the force formula without redesigning the rule breaks the system. Closing the long-horizon gap appears to require joint redesign, not a drop-in bootstrapping patch; neither variant ships in the released code.

7 Conclusion

We formalized NWM, a potential-field, non-parametric RL framework, and evaluated it against Random, tabular Q-learning, and DQN on three classic-control tasks with a reproducible, seed-controlled protocol. The study clarifies the operating regime of instance-based potential-field control and provides an honest, open baseline. Future work includes learned similarity/embeddings for higher-dimensional inputs, bootstrapped score propagation to strengthen credit assignment, and hybrid schemes that use the repulsive field as a safety layer around a parametric policy.

Reproducibility Statement

The library, benchmark harness, exact seeds, hyperparameters, and the scripts that regenerate every table and figure in this paper are released at `https://github.com/CastermustOfficial/NWM`. Running the benchmark and `paper/make_paper_assets.py` regenerates `tables/results_table.tex` and the figures used above.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[2] Charles Blundell, Benigno Uria, Alexander Pritzel, Yazhe Li, Avraham Ruderman, Joel Z Leibo, Jack Rae, Daan Wierstra, and Demis Hassabis. Model-free episodic control. In *arXiv preprint arXiv:1606.04460*, 2016.

[3] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.

[4] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2018.

[5] Oussama Khatib. Real-time obstacle avoidance for manipulators and mobile robots. *The International Journal of Robotics Research*, 5(1):90–98, 1986.

[6] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3):293–321, 1992.

[7] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[8] Alexander Pritzel, Benigno Uria, Sriram Srinivasan, Adrià Puigdomènech Badia, Oriol Vinyals, Demis Hassabis, Daan Wierstra, and Charles Blundell. Neural episodic control. In *International Conference on Machine Learning (ICML)*, pages 2827–2836, 2017.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[10] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

[11] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3):279–292, 1992.

[12] B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962.